Building Knowledge Graphs: From Sources to Verifiable Queries

Mick Cooney

9 November 2026

Table of Contents

1 Introduction **2**

2 Define the Boundary Before Building the Graph **2**
2.1 Bounded Source Claim Construction 5

3 Preserve Mentions, Entities, and Identifiers as Different Things **5**

4 Model Qualified Claims and Source Assertions **7**

5 Extract Conservatively and Resolve Identity with Abstention **8**

6 Provenance, Time, Contradiction, and Uncertainty **10**

7 Privacy and Proportionate Identity **11**

8 Construct and Query with Python and DuckDB **12**
8.1 From Fixture to Tables 13
8.2 Direct and Two-Hop Questions 15
8.3 Valid Time, Conflict and Evidence 16

9 Represent Claims in RDF and Property Graphs **17**
9.1 RDF: Triples and Qualified Claims 18
9.2 SPARQL Evidence Queries 20
9.3 Validate Graph Structure with Shapes Constraint Language (SHACL) 22
9.4 Describe Production History with the PROV Ontology (PROV-O) . . 23
9.5 Property Graphs and Cypher 24
9.6 What Is Lost When a Graph Is Simplified? 25
9.7 Storage and Workload Determine the Boundary 27
9.8 A Bounded Future Neo4j Smoke Test 29

10 Analyse a Projection in R **30**

11 Validate, Evaluate, and Diagnose **30**
11.1 Validation, Inference and Verification 31
11.2 Gold Sets and Useful Measures 32

12 Release, Update, and Govern **34**

13 Further Reading **36**

1 Introduction

A knowledge graph connects identified things and the relationships claimed between them. Its value is not that drawing an edge makes a statement true. Its value is that a reader can follow a chain from a question through explicit modelling decisions to the source records on which an answer depends. This primer develops that chain: from source material and uncertain names to qualified claims, evidence-backed queries, validation, and repeatable releases.

The reader is a software or data practitioner comfortable with tables, JSON, and basic Python or SQL. No prior graph-database or ontology experience is assumed. The main route uses a small, synthetic organisational-policy example and a canonical Python and DuckDB construction pipeline. Later sections compare the same conceptual model with the Resource Description Framework (RDF), property graphs, and R graph analysis. This is a construction guide, not a complete reference for database products, formal logic, or legal interpretation. Policy authority in the example is an explicit modelling assumption, not legal advice.

The standards, DuckDB, Neo4j and repository-runtime claims are linked to primary sources. The source-to-claim example is deliberately bounded, and research and privacy claims are linked or flagged with their evidential limits. The graph-query, shape-validation and provenance snippets are illustrative and unexecuted.

2 Define the Boundary Before Building the Graph

A knowledge graph is an answer-producing model over a declared evidence set. Its boundary therefore includes more than the files that happened to be ingested. It includes the questions the model is intended to answer, the source classes it accepts, the period and population those sources cover, and the decisions it is

authorised to make when records disagree. If any of these are implicit, users can mistake a bounded extract for a complete account.

Northbridge makes the boundary concrete. The fixture contains an authorised policy register, a Northbridge directory, and Atlas meeting minutes. It represents two policy versions, two projects, Morgan Lee, and two distinct people named Alex Kim. For the teaching scenario, the register is treated as the designated source for recorded policy applicability and ownership; the directory supports project affiliation; the minutes represent what a meeting record states. This is a modelling assumption, not a source-ranking rule implemented by the script. Those roles do not make the register universally authoritative about every question. The minutes are the primary evidence for the sentence recorded at the meeting, even where the register is preferred to answer which version the organisation recorded as applicable.

Competency questions turn that boundary into observable requirements. For example: which policy version did the register say applied to Atlas on 1 March 2024? What owner did the register give for that version? What did the 15 March minutes say, and did that statement agree with the register for the same date? Can the attendee called Alex Kim be tied to either directory person? Each question names a relation or answer, and should also settle relevant scope: source authority, temporal interpretation, required evidence, and what to do when evidence is insufficient. A question that asks "Who is responsible?" leaves open whether responsibility means policy ownership, project sponsorship, or meeting attendance.

Treat each question as a contract for a query and its result. A useful answer to the first Northbridge question includes the version, the date tested, the authority rule, and a source locator. An answer to the attendee question may be "the minutes name Alex Kim, but the fixture does not distinguish the two directory people." This is a successful outcome if the evidence does not support a unique link. A graph that always returns a person has encoded an assumption, not demonstrated better coverage.

The boundary also governs negative answers. The absence of a matching claim means only that no claim was found in this fixture after applying the query's filters. It does not establish that Northbridge has no other policies, that no other meeting discussed the rule, or that Atlas has no other affiliations. To interpret absence as a negative fact, the source set needs a completeness guarantee for

a defined subject, predicate, period, and population. The small synthetic fixture offers no such guarantee. Its appropriate use is to demonstrate construction, conflict handling, and query behaviour on known records, not to estimate coverage of a real organisation.

This distinction prevents two common category errors. First, "the graph says" should not conceal which source and selection rule produced the result. Second, a missing path is not evidence that a relationship does not exist. State the fixture boundary alongside conclusions, and return "not represented in the supplied sources" when that is all the query establishes.

The fixture is `primer_knowledge_graph/examples/fixture.json`. It is synthetic data, not a set of retained source documents. The table summarises its source records and selected assertions; the locators are the exact source-level labels in the JSON.

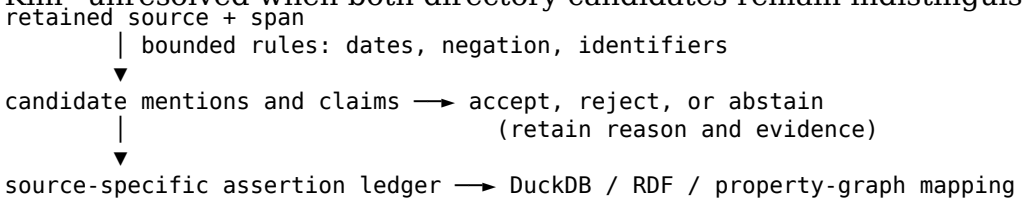
Source	Locator	Selected fixture content
Authorised policy register `s-register`	`R1:P1-P2`	`c1`: version 1 applies to Atlas from 1 January to 1 April 2024; `c2`: version 1 is owned by Morgan; `c6`: version 2 applies from 1 April.
Northbridge directory `s-directory`	`D1:A1`	`c3` and `c7`: Atlas and Orion belong to Northbridge. The shared source locator does not identify a separate Orion row.
Atlas meeting minutes `s-minutes`	`M1:paragraph-3`	From 15 March, `c4` asserts version 2 applies and `c5` denies that version 1 applies; `m-alex` is unresolved.

The JSON stores the two Alex entities but does not contain their directory rows or the full register and minutes. It records only version 1 ownership, not ownership

of version 2. The start date on a meeting claim is an encoded valid-time interpretation, not a general rule that a publication date automatically fixes validity.

2.1 Bounded Source Claim Construction

The construction decision is easiest to see in a small, deterministic slice. Suppose the retained minutes contain the sentence “From 15 March, version 2 applies to Atlas; version 1 does not.” Keep the source version and the exact sentence span, then produce two candidate mentions and two qualified claims: a positive `applies_to` claim for `policy:travel-v2`, and a negative claim for `policy:travel-v1`. The meeting’s explicit “from 15 March” supplies `valid_from`; it does not, by itself, supply an end date or prove continuation after the observed statement. A rule records the negation and a resolver leaves “Alex Kim” unresolved when both directory candidates remain indistinguishable.



This is a worked construction boundary, not a claim that the repository contains a general extractor. The runnable Python example applies the bounded rules to retained synthetic source records, compares the constructed `c1`, `c4`, and `c5` claims with the canonical ledger, and then loads the ledger; the remaining claims are deliberately curated. Full-document optical character recognition, open-domain language understanding, learned matching, and a general update engine remain outside this milestone.

3 Preserve Mentions, Entities, and Identifiers as Different Things

A mention is an occurrence in a source; an entity is a thing the model refers to; an identifier names that entity within a defined namespace. A label is a human-readable description and need not be unique. These objects have different lifecycles and should not be collapsed into one row just because a simple graph drawing has one node per thing.

In the fixture, `m-alex` records “Alex Kim” in `s-minutes` with a null entity. Two distinct entities, `person:alex-kim-1` and `person:alex-kim-2`, share that label.

Resolution decision `r2` records both candidates and abstains because the meeting gives no distinguishing identifier. The occurrence survives without inventing an identity or a third person.

The same separation applies to policy names. `policy:travel-v1` and `policy:travel-v2` are distinct version entities. The mentions `m-policy-v1`, `m-policy-v1-meeting`, `m-policy-v2-meeting`, and `m-policy-v2-register` are separate textual occurrences referring to those versions. A mention such as "version 2" is meaningful because its source context supplies a policy subject; outside that context the words alone may be ambiguous. Modelling the policy family as though it were identical to either version would blur the temporal question the fixture is designed to answer.

Each source has a logical locator: `R1:P1-P2`, `D1:A1`, or `M1:paragraph-3`. These identify synthetic source regions, not byte offsets or retained source documents; the register locator covers both rows rather than singling out one claim. A production evidence chain should point to a stable source version and a reproducible row key, page and paragraph, or character offsets against a content-addressed text extraction. Offsets without a reproducible text version are fragile; row numbers are fragile if rows can be sorted.

Identifiers should be stable and namespaced. The fixture's IDs (`org:northbridge`, `project:atlas`, `policy:travel-v1`, and `person:morgan-lee`) are readable synthetic identifiers; the prefix suggests a kind but does not by itself guarantee global uniqueness or establish who issued it. Real source identifiers should retain their issuing authority, since `1234` in one system may identify something different from `1234` in another. A label change should not silently create a new entity, and a reused label should not silently merge two entities. If two records are linked, preserve the mapping and the basis for it so that a later correction can split the records without rewriting the source history.

The World Wide Web Consortium (W3C) RDF 1.1 Concepts recommendation makes a related distinction between Internationalised Resource Identifiers (IRIs), literals, and blank nodes: identifiers and values play different roles in a graph. Northbridge's JSON schema is not RDF, but the conceptual lesson applies across storage choices. A date such as `2024-03-15` is a value; `project:atlas` is a reference; "Atlas" is a label; and the occurrence `m-atlas` is evidence that a particular source used that label. Retaining these distinctions allows labels and interpretations to change without losing what the source

actually contained.

4 Model Qualified Claims and Source Assertions

A bare triple such as `policy:travel-v1 applies_to project:atlas` omits information needed to answer the Northbridge questions. The fixture's claim `c1` adds source `s-register`, mention `m-policy-v1`, the validity interval from 1 January to 1 April 2024, and positive polarity. These fields make the statement inspectable and temporally queryable. The corresponding positive relation in `relations` is only an analytical projection; it omits source, mention, interval, and polarity and must not become the evidence record used to explain an answer.

The subject–predicate–object form remains useful for the core relation, but many claims are not adequately represented by an unqualified edge. A policy's applicability can be conditional on jurisdiction, staff class, or activity; ownership can have a role and appointment period; an employment relation can be asserted by one source and disputed by another. One representation is to make each qualified assertion a first-class record, as `claims` does, with references to its participants and evidence. Another is an n-ary relation node or a named graph with attached metadata. The design choice should preserve the fields needed by competency questions and validation. W3C's guidance on n-ary relations describes this general modelling problem; it does not prescribe one universal encoding.

Distinguish a proposition from an assertion of that proposition. In this primer, a claim is the qualified proposition under consideration; an assertion is a source or process presenting it. Claim `c4` says that `policy:travel-v2` applies to Atlas from 15 March according to the minutes. Claim `c6` says that the same version applies from 1 April according to the register. Claim `c5` records the minutes' negative assertion that version 1 applies on 15 March. Keeping source-specific assertions means the minutes and register remain separately queryable. It also avoids the misleading operation of overwriting one value with the other.

The temporal contract must distinguish three end kinds; nullness alone is not enough. A `bounded` claim has `valid_to > valid_from` and means the half-open interval `[valid_from, valid_to)`. An `unbounded` claim has no `valid_to` and explicitly says that the source asserts continuation; it remains revisable and is not a promise of eternal truth. An `unknown` claim has no `valid_to` but has a

`supported_through` date: the source supports the inclusive period `[valid_from, supported_through]`, and applicability afterwards is indeterminate. `supported_through` is the last domain-valid date supported by the assertion, not its publication, ingestion, review, or transaction time. It must not be inferred from a single observation or from a source publication date.

Thus `c1` is bounded, covers 1 January through 31 March, and `c6` begins on 1 April. Existing claims with an unspecified end should be treated as `unknown` unless the source explicitly supplies continuation; do not silently promote them to `unbounded`. The fixture and runnable script require explicit `end_kind`, explicit nulls where applicable, and the corresponding `supported_through` rule. The predicate itself also needs a definition: `owned_by` means the register's recorded owner in this example, not necessarily the legal owner, budget holder, or operational decision-maker.

Polarity is not the same as truth status. `c5` is a negative-polarity assertion from the minutes; its presence records that the minutes deny applicability of version 1 from 15 March. It does not erase positive `c1`, and the fact that `c1`'s validity interval ends on 1 April does not make `c5` true. The positive-claim `relations` projection intentionally excludes `c5`, so analyses over that projection cannot answer questions about the disagreement. Query design must choose whether it needs the positive topology, the full assertion ledger, or both. A projection is a derived view with documented loss, not a replacement for source-backed claims.

5 Extract Conservatively and Resolve Identity with Abstention

Extraction turns source material into candidate mentions and assertions. It is an interpretation step, even when a rule-based parser is used: sentence boundaries, negation scope, coreference, version references, and date expressions can all be misread. Preserve the input source and version, the extracted text or span, the proposed participants and predicate, and the extraction method. Separate this candidate record from a later accepted assertion so that changed rules can be compared against the same evidence.

The Northbridge examples expose why this matters. The register says version 1 applies to Atlas from 1 January until 1 April; the minutes, dated 15 March, say version 2 already applies and explicitly deny that version 1 applies. A system

that extracts only affirmative noun–verb patterns may retain `c4` and lose `c5`. A system that treats the meeting date as the start date of the policy's validity may confuse when a statement was made with when the stated relationship holds. The fixture encodes the meeting assertions as valid from 15 March, but a richer representation would separately store the minutes' publication or event date and the time referred to by each statement. Extraction must retain enough context to assess those interpretations rather than hide them in a single edge.

Entity resolution is a separate decision: given a mention or source record and candidate entities, decide whether the evidence supports identity. Candidate generation and candidate matching have distinct failure modes. A blocking rule may fail to propose a true match; a matcher may incorrectly accept a proposed pair. Statistics Canada's study of blocking errors discusses why missed candidates need separate measurement from match decisions. Names, type compatibility, co-occurring identifiers, dates, and source-specific rules can help rank or reject candidates, but no string similarity threshold supplies evidence that is absent from the source.

Resolution decision `r1` accepts `m-morgan` as `person:morgan-lee`, giving a matching issuing-directory identifier as its reason. The identifier itself and the underlying directory record are not present as fields in the fixture, so this is a recorded synthetic decision, not an independently demonstrated match. In contrast, `r2` abstains on Alex with two candidates. A type mismatch could justify rejecting a candidate; insufficient evidence calls for abstention.

The fixture records each decision ID, mention, candidate set, outcome, selected candidate where applicable, and reason. It contains one acceptance and one abstention, but no rejection. An operational decision record should also identify the method and version, review status and decision time. The Python validator checks the decision references and selected-candidate rules, and asserts the Alex abstention; it does not run a matcher or write the decisions into DuckDB.

Abstention is useful because the costs of errors are asymmetric. A missed link may leave a query incomplete; a false identity merge can attribute meeting attendance or policy ownership to the wrong person and contaminate every downstream path. Preserve uncertain candidate links outside the accepted graph, and make review possible. Pairwise decisions also need global consistency checks: accepting A–B and B–C does not prove A–C if the evidence is inconsistent or entities are not governed by a strict equivalence relation.

Record enough provenance to reverse or revisit a merge.

Do not interpret a model's score as a probability of real-world truth unless it has been calibrated and evaluated for that task and population. Keep uncertainty tied to its source: ambiguous name, uncertain date, source reliability, incomplete coverage, or extraction disagreement. Evaluation should report candidate-generation recall separately from accepted-match precision and review/abstention rates, using an explicitly described labelled sample. The synthetic fixture is a behavioural example; it cannot establish those rates for Northbridge or any wider population.

6 Provenance, Time, Contradiction, and Uncertainty

Provenance makes an answer auditable: which source version supported which assertion, what activity extracted or reviewed it, and which release and selection rule produced the answer. The W3C PROV Ontology (PROV-O) vocabulary models entities, activities, and agents and can inform an implementation. Even without adopting PROV-O, preserve stable source identifiers, version or digest, source and ingestion dates where known, reproducible locator, extraction or review method, and the responsible process or agent. A source publisher, the person quoted, and the extractor are different roles and should not be collapsed.

Time commonly has at least two axes. Valid time describes when the asserted relationship holds in the domain; transaction or observation time describes when the system recorded that assertion. The fixture's `valid_from` and `valid_to` encode valid time, while its `sources[].published` gives source publication dates. It has no ingestion timestamp or transaction history, so it cannot answer "what had the system recorded by 20 March?" without an external release record. A corrected register imported on 1 May should not rewrite the evidence available in an earlier release. Preserve both the old and new source versions and state whether a historical query asks what was known then or what current evidence says about then.

For the date 15 March, the register's `c1` covers Atlas under version 1; the minutes' `c4` asserts version 2 and `c5` denies version 1 for the same project and date. Those are incompatible assertions under the fixture's intended interpretation. `c6` begins version 2 on 1 April and is consistent with the end of `c1`; it is not itself a contradiction. Before labelling two records contradictory, compare

subject, predicate, object or competing value, temporal scope, jurisdiction, and relevant conditions. A conflict requires overlapping scope and mutually exclusive claims, not merely two different records.

Authority is purpose-specific. To answer “what did the register record?”, use the register. To answer “what did the minutes say?”, report the minutes. To answer “which version should a user treat as governing?”, the fixture alone cannot settle legal or operational force; its teaching rule can prefer the register only for the narrower question about recorded policy applicability. Keep the losing assertion visible and state the preference rule. Otherwise a source ranking can look like a truth mechanism and conceal a material disagreement.

Uncertainty has multiple causes and should not be compressed into one confidence number. The Alex identity is underdetermined because the source lacks a distinguishing identifier. The applicability disagreement is not an extraction uncertainty in the fixture: two explicit, incompatible records are present. Source completeness is unknown because only three synthetic source records are included. An unknown end on `c2` means duration is supported only through its retained horizon; it is not an eternal interval. Each condition changes what a query can conclude and should be represented or reported separately.

Agreement between two assertions is not automatically independent corroboration. They may copy the same upstream register, share an author, or be generated from one database. Provenance can reveal common origin; counting edges cannot. Similarly, a path from a policy through a project to an organisation proves only that the selected projection contains those edges. It does not establish causality, authority, or completeness. Return evidence for each hop and keep interpretation distinct from topology.

7 Privacy and Proportionate Identity

Entity resolution can turn dispersed records into a more identifying profile than any single source. A name, project affiliation, and meeting attendance may be low-risk in isolation but sensitive when linked and made searchable. The Northbridge fixture is fictional, yet its two Alex Kim records demonstrate a practical harm: an unsupported merge could attribute attendance to the wrong person. In a real system, that same error can affect access decisions, employment records, or investigations.

Collect and expose only the identifiers and attributes required by the competency questions. If a policy query needs the owner relation, it may not need personal contact details or a broad employee profile. Keep direct identifiers in a controlled mapping where possible, apply access controls to source text and derived claims, and record retention and deletion rules. A graph edge is still personal data when it identifies or can be linked to an individual; changing the storage format does not remove that responsibility. The appropriate legal basis, safeguards, and retention period depend on jurisdiction and purpose and require review against applicable law and organisational policy.

Privacy also affects provenance design. A public answer may need to show a source class, version, and redacted locator rather than expose an entire restricted document. The system should preserve enough controlled evidence for authorised audit while preventing broader access through graph traversal. Test both what an authorised reviewer can trace and what an ordinary query user can infer. Access restrictions and redaction must not silently erase the fact that evidence was withheld; report the limitation at a level that does not itself disclose protected information.

The practical design test is whether each identity link and exposed attribute is necessary for a stated question, supported by evidence, and available to the intended audience. For the ambiguous Northbridge attendee, abstaining protects both correctness and the people represented. It also leaves a clear route for adding a distinguishing identifier later, should an authorised source provide one.

8 Construct and Query with Python and DuckDB

The canonical runnable example is `primer_knowledge_graph/examples/python/knowledge_graph` its inputs are `primer_knowledge_graph/examples/source_records.json` and the canonical `primer_knowledge_graph/examples/fixture.json`. Both use synthetic records. The script applies the bounded construction slice, compares constructed claims with the ledger, validates references and intervals, writes claims and provenance to a file-backed DuckDB database, closes it, reopens it read-only, and runs bounded queries with assertions. It is not a general source extractor. Its manifest entry is `knowledge-graph-python` in `primer_knowledge_graph/examples/manifest.toml`. From the repository root,

build the primer's runtime if necessary and run:

```
just build-python-duckdb-runtime
just examples knowledge_graph knowledge-graph-python
```

The runner provides a fresh temporary output directory, disables network access and discards the database afterwards. The script uses `EXAMPLE_OUT` when set and otherwise writes to its working directory. It creates tables with plain `CREATE TABLE`, so running it again against the same output database fails rather than silently replacing the previous build. Use a fresh directory for each standalone run. DuckDB documents file-backed and read-only connections; persistence here means the script can reopen and query its file, not that it publishes an enduring release.

The fixture contains eight entities, three source records, seven mentions and seven claim records. It also contains five `relations` for a separate analytical projection. The Python script validates entity references but does **not** create an entities or relations table. It writes only `claims`, `sources` and `mentions`. This is enough to demonstrate evidence-bearing joins, but an application needing entity labels, types, and database-enforced references must add those tables and load them. The R example independently hard-codes the same entity IDs and projection for analysis; the projection is not the canonical evidence store. The retained source records used by the construction slice are a separate synthetic input and include a deliberately refused date phrase.

8.1 From Fixture to Tables

In the target ledger, each JSON claim is a source-specific assertion with an ID, subject, predicate, object, source, optional mention, valid interval, temporal end kind, optional support horizon, and polarity. The field contract is: `bounded` requires `valid_to` and no `supported_through`; `unbounded` requires both to be null and explicit source wording of continuation; `unknown` requires a null `valid_to` and `supported_through >= valid_from`. Dates use calendar-date strings. The horizon is the last supported domain-valid date, inclusive; it is not publication or ingestion time. The checked-in fixture uses this contract. In either form, `c1` says that `policy:travel-v1` applies to `project:atlas` from 1 January to 1 April 2024, according to `s-register`; `c5` records a negative assertion about the same subject, predicate and object from `s-minutes`, starting on 15 March. The two rows preserve disagreement. `c4` and `c6` both assert that version 2 applies to Atlas, but they have different sources and start dates. Neither should be collapsed

into a single source-free edge if the question concerns evidence.

The executable's current data-definition language (DDL) is intentionally minimal; domain rules remain in the Python validator:

```
CREATE TABLE claims (id VARCHAR, subject VARCHAR, predicate VARCHAR,
    object VARCHAR, source VARCHAR, mention VARCHAR, valid_from DATE,
    valid_to DATE, end_kind VARCHAR, supported_through DATE,
    polarity BOOLEAN);
CREATE TABLE sources (id VARCHAR, title VARCHAR, published DATE, locator VARCHAR);
CREATE TABLE mentions (id VARCHAR, source VARCHAR, text VARCHAR, entity VARCHAR);
```

The executable loads both qualifier fields and validates the three combinations before insertion. A production-aligned schema would also make `end_kind` non-null, add keys and foreign keys, and enforce the cross-field combinations in the database as well as in application code.

Loading uses parameterised `executemany` calls, with columns selected in the same order as the DDL. The JSON dates are converted to DuckDB `DATE` values on insertion. Prepared values keep data separate from SQL syntax; they do not turn a dynamic table or column name into a safe identifier. See the DuckDB Python DB-API documentation for parameter binding. The executable validates before insertion, rather than relying on the DDL for uniqueness, required values or foreign keys.

The example's DDL does not enforce keys, required fields or foreign keys. The Python validator checks several of these before insertion, including that a claim's mention comes from its source and identifies a participant. A more defensive database schema could add primary and foreign keys, non-null fields and interval checks; cross-row conditions and domain rules would still need additional validation. Consult DuckDB constraints for the pinned version's behaviour.

The source table includes the fixture's logical locator, but the fixture has no retained source version, digest, ingestion time or complete source document. A production system should add these and use assertion-to-evidence links where one assertion depends on several passages. Evidence links can share a source and are not necessarily independent corroboration.

The distinction between a proposition and an assertion matters when several records say the same thing. This fixture uses `claims` as source-specific rows: the source and interval are part of each row. A larger model may instead give a canonical proposition its own ID and record each source assertion separately, including its extraction method and review state. Either design can work if it

preserves which source made which assertion, under which scope, and does not mistake a merged proposition for a verified fact. The W3C PROV data model provides a general vocabulary for entities, activities and agents when interoperable provenance is needed; the fixture does not implement PROV.

8.2 Direct and Two-Hop Questions

A direct query returns all recorded assertions for a subject and predicate. In the executable, the parameters are `policy:travel-v1` and `applies_to`:

```
SELECT id, object, polarity
FROM claims
WHERE subject = ? AND predicate = ?
ORDER BY id;
```

The result is `c1, project:atlas, true` and `c5, project:atlas, false`. The query deliberately has no date or source filter. It establishes that the database contains two differently signed assertions; it does not decide which applies on a particular day. Filtering to positive rows first would hide the negative evidence. Filtering by `source = 's-register'` answers a different, explicitly source-scoped question, not an unrestricted question about all evidence.

A two-hop query follows a known relationship sequence. The executable joins the object of a positive `applies_to` claim to the subject of a positive `belongs_to` claim:

```
SELECT a.id, b.id, b.object
FROM claims a JOIN claims b ON a.object = b.subject
WHERE a.subject = ? AND a.predicate = ? AND b.predicate = ?
  AND a.polarity AND b.polarity
ORDER BY a.id, b.id;
```

With parameters `policy:travel-v1`, `applies_to` and `belongs_to`, the result is `(c1, c3, org:northbridge)`: the policy is linked through Atlas to Northbridge. Both claim IDs are returned, so the evidence for each step can be inspected. The join establishes a path under the query's conditions; it does not create a new assertion that the policy applies to the whole organisation. The executable's two-hop query has no time filter and therefore does not establish that both links hold on any requested date. For a dated path, apply the same valid-time predicate to **each** row, and retain the individual intervals and sources. If several assertions support either step, the join can yield several paths; deciding whether to deduplicate answers is separate from retaining their evidence.

Bounded predicates and hop counts also constrain cost and interpretation. An unrestricted traversal may find a connection through irrelevant relationship

types, combine incompatible dates, or return many paths created by duplicated evidence. State which predicate sequences answer the question and why their direction matters. A path through `owned_by` has a different meaning from one through `belongs_to`, even if both lead to a familiar name.

8.3 Valid Time, Conflict and Evidence

The target query policy classifies each matching assertion before aggregating an answer. A bounded end is exclusive; an explicit unbounded end remains applicable after its start; an unknown end is applicable only through its inclusive support horizon and is indeterminate afterwards. The classification is:

```
SELECT id, polarity, end_kind, supported_through,
  CASE
    WHEN CAST(? AS DATE) < valid_from THEN 'outside'
    WHEN end_kind = 'bounded' AND CAST(? AS DATE) < valid_to THEN 'applicable'
    WHEN end_kind = 'bounded' THEN 'outside'
    WHEN end_kind = 'unbounded' THEN 'applicable'
    WHEN end_kind = 'unknown'
      AND CAST(? AS DATE) <= supported_through THEN 'applicable'
    WHEN end_kind = 'unknown' THEN 'indeterminate'
    ELSE 'invalid'
  END AS temporal_status
FROM claims
WHERE subject = ? AND predicate = ? AND object = ?
ORDER BY id;
```

The source-specific answer policy is deliberately conservative: conflicting applicable polarities produce `abstain: conflicting evidence`; any indeterminate row produces `abstain: indeterminate temporal scope`; one applicable polarity produces `supported` or `refuted`; and no applicable or indeterminate evidence produces `abstain: no evidence`. This is a teaching policy, not the only valid aggregation rule. Do not turn an unknown negative into a definite refutation, or discard it to obtain a definite support. The executable applies this policy after retaining each row's temporal classification.

Under this contract, 31 December 2023 has no evidence; 15 March has applicable `c1` and `c5` and therefore a conflict; 16 March has applicable `c1` and indeterminate `c5`; and 1 April and 15 April have an outside `c1` and indeterminate `c5`. These are classifications relative to the selected evidence and policy, not claims that the world changed at an unobserved time. They do not establish when a reader could first have known each statement. For that, record when each assertion became available to the system, preserve its changes and filter both valid time and knowledge time. Publication date alone is not a substitute for ingestion or transaction history.

Here `refuted` means that the selected applicable rows contain negative evidence and no positive evidence under this policy. It is not a judgment that the negative source is authoritative or that every other relevant record has been found. No evidence is different from explicit negative evidence. Nor does a conflict force a particular authority decision: the policy does not rank the register above the minutes.

A provenance query makes that disagreement inspectable:

```
SELECT c.id, s.title, s.locator, m.text
FROM claims c
JOIN sources s ON c.source = s.id
LEFT JOIN mentions m ON c.mention = m.id
WHERE c.subject = ? AND c.predicate = ? AND c.object = ?
ORDER BY c.id;
```

The executable returns `c1` with `Authorised policy register`, `R1:P1-P2` and `Travel Policy v1`, and `c5` with `Atlas meeting minutes`, `M1:paragraph-3` and `version 1`. The left join matters because a claim such as `c7` has no mention. These source-level locators are logical labels, not the underlying documents or verified quotations. A real system needs source versions, reproducible locations, access conditions and retrieval context.

A dated query for version 2 illustrates a separate issue: `c4` is positive from 15 March and `c6` is positive from 1 April. The presence of two positive rows after 1 April does not prove independent corroboration or settle the register-versus-minutes interpretation. If an application adopts an authority hierarchy, make that policy a versioned input to answer generation, retain excluded evidence, and explain why it applies to this predicate, scope and date. A later correction must be able to change the answer without erasing the historical disagreement.

9 Represent Claims in RDF and Property Graphs

A representation is useful when it preserves the distinctions needed to answer a question, not merely when it draws a convincing network. Follow the Northbridge register's version 1 applicability statement through each representation: the policy version applies to Atlas from 1 January 2024 up to, but excluding, 1 April; the assertion comes from register `R1`, locator `R1:P1-P2`. The meeting minutes make a competing statement about version 2 on 15 March. A representation that retains only two policy-to-project arrows cannot reconstruct the dates, the source of either statement, or why the statements conflict. Nor should it turn the unresolved meeting mention of Alex Kim into an identified person.

The examples in this section are illustrative designs inspired by the fixture. Their identifiers and vocabulary are local to the examples; they are not a claim that a particular RDF export, property-graph adapter, or Neo4j service has been implemented. In particular, distinguish a *claim*, which describes a qualified proposition, from an *assertion*, which records a source or process presenting that proposition. A source can support several claims, and several assertions can refer to the same claim while retaining separate locators and histories.

9.1 RDF: Triples and Qualified Claims

The Resource Description Framework (RDF) represents a graph as a set of triples: subject, predicate, and object. A triple says that the predicate relates the subject to the object. The subject is an IRI or blank node, the predicate an IRI, and the object an IRI, blank node, or literal. An IRI is an identifier in a global naming system; a literal is a value with a datatype or language tag. Blank nodes identify resources without providing a globally meaningful IRI. A graph is a *set*, so repeating the same triple does not record a second independent source. See RDF 1.1 Concepts for the abstract model.

For example, the Turtle statement `ex:atlas ex:belongsTo ex:northbridge .` is a readable serialisation of one triple. Prefix `ex:` abbreviates a namespace, rather than defining a separate kind of identifier. Turtle's punctuation lets multiple triples share a subject or predicate. Two serialisations with different ordering can describe the same RDF graph, and the spelling of a blank-node label in one file is not a stable identifier to use across releases. A persistent source locator or claim that must be cited across imports is therefore better given a governed IRI than identified by a local blank-node label.

A single direct triple such as `ex:travelV1 ex:appliesTo ex:atlas` does not contain the interval or source. Putting such a triple in a graph also asserts that relation in that graph; the existence of an assertion *about* the relation is a different modelling choice. For the conservative temporal contract, map `valid_from` and `valid_to` to `ex:validFrom` and optional `ex:validTo`; put exactly one `ex:endKind` on the claim using `ex:BoundedEnd`, `ex:UnboundedEnd`, or `ex:UnknownEnd`; put `ex:supportedThrough` on the source-specific assertion; and put required `ex:polarity` on the qualified claim. Use `xsd:dateTime` at midnight Coordinated Universal Time (UTC) for these domain dates so portable comparisons in the SPARQL Protocol and RDF Query Language (SPARQL) use

one ordered datatype; this is an encoding convention, not invented time-of-day precision. Model the qualified proposition as its own resource instead:

```
@prefix ex:   <https://example.org/northbridge/> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix prov: <http://www.w3.org/ns/prov#> .

ex:claim-v1-atlas a ex:ApplicabilityClaim ;
    ex:policyVersion ex:travelV1 ;
    ex:project ex:atlas ;
    ex:validFrom "2024-01-01T00:00:00Z"^^xsd:dateTime ;
    ex:validTo "2024-04-01T00:00:00Z"^^xsd:dateTime ;
    ex:endKind ex:BoundedEnd ;
    ex:polarity true .

ex:assertion-r1-p1 a ex:Assertion, prov:Entity ;
    ex:asserts ex:claim-v1-atlas ;
    ex:sourceRecord ex:r1 ;
    ex:locator "R1:P1-P2" .

ex:claim-v1-atlas-minutes a ex:ApplicabilityClaim ;
    ex:policyVersion ex:travelV1 ;
    ex:project ex:atlas ;
    ex:validFrom "2024-03-15T00:00:00Z"^^xsd:dateTime ;
    ex:endKind ex:UnknownEnd ;
    ex:polarity false .

ex:assertion-minutes-p3 a ex:Assertion ;
    ex:asserts ex:claim-v1-atlas-minutes ;
    ex:sourceRecord ex:minutes ;
    ex:locator "M1:paragraph-3" ;
    ex:supportedThrough "2024-03-15T00:00:00Z"^^xsd:dateTime .
```

The local vocabulary specifies that `validTo` is exclusive; RDF does not infer that convention from its name. The code expresses a claim resource and an assertion resource; it deliberately does not add the unqualified direct `appliesTo` triple. The assertion's source, locator and support horizon make the example traceable at the fixture's source-level logical granularity. The unknown-ended minutes claim is not applicable after its horizon; it is indeterminate. A production record also needs the retained source version, a reproducible locator, and any extraction or review information required for an audit. Here `ex:r1` is an identifier for the register record, not the literal bytes of the source. These snippets are illustrative and unexecuted.

This is an n-ary relation: the application of a version to a project depends on more than the two endpoints. The claim resource makes the period and other conditions participants in one identified description. If a second source independently reports the *same* version, project, period and conditions, it can have a second assertion pointing to that claim. If it states a different interval or version, create a distinct claim. The W3C guidance on n-ary relations presents

this family of patterns. Claim identity and deduplication remain application decisions; sharing an endpoint pair is not enough to establish that two claims are identical.

RDF datasets can organise a default graph and named graphs. A named graph can group an export or source-specific statements, but its name alone does not automatically say who asserted each statement, when it was obtained, or whether it is authoritative. Publish those semantics and provenance explicitly. Grouping by source may help access or retrieval, but one source can contain several locators and claims, and one assertion may rely on multiple source fragments. Named graphs do not remove the need to model that detail when a question requires it.

The examples use RDF 1.1 constructs for portability. Other statement-annotation facilities exist, but a deployment must specify the exact RDF version, syntax, semantics, and store support before depending on them. Even then, annotation syntax does not decide whether two observations are independent, which source takes precedence, or whether a date has been interpreted correctly.

9.2 SPARQL Evidence Queries

The SPARQL Protocol and RDF Query Language (SPARQL) matches graph patterns and returns bindings for variables. Each triple pattern below must match; the shared variables join the claim to its assertion and source. For the illustrative data above, this query retrieves the register claim applicable on 1 March, including its locator:

```
PREFIX ex:  <https://example.org/northbridge/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?claim ?version ?assertion ?source ?locator ?start ?end
       ?kind ?polarity ?supportedThrough ?temporalStatus
WHERE {
  ?claim a ex:ApplicabilityClaim ;
         ex:policyVersion ?version ;
         ex:project ex:atlas ;
         ex:validFrom ?start .
  ?assertion ex:asserts ?claim ;
             ex:sourceRecord ?source ;
             ex:locator ?locator .
  ?claim ex:endKind ?kind ; ex:polarity ?polarity .
  OPTIONAL { ?claim ex:validTo ?end }
  OPTIONAL { ?assertion ex:supportedThrough ?supportedThrough }
  BIND ("2024-03-01T00:00:00Z"^^xsd:dateTime AS ?day)
  BIND (IF(?day < ?start, "outside",
```

```
       IF(?kind = ex:BoundedEnd,
          IF(BOUND(?end) && ?day < ?end, "applicable", "outside"),
       IF(?kind = ex:UnboundedEnd, "applicable",
       IF(?kind = ex:UnknownEnd && BOUND(?supportedThrough) &&
             ?day <= ?supportedThrough, "applicable",
       IF(?kind = ex:UnknownEnd, "indeterminate", "invalid")))))
       AS ?temporalStatus)
  FILTER (?source = ex:r1)
}
ORDER BY ?version ?assertion
```

The `BIND` classifies evidence rather than treating an unbound end as open. A query that returns only `?temporalStatus = "applicable"` is a deliberately narrower register-selection query; an answer-producing query must retain `indeterminate` rows for conservative aggregation. The source filter asks the narrower question of what the register records; removing it could expose both register and minutes claims, provided both have been loaded with comparable dates. To answer what the meeting said, select the minutes and their locator rather than changing the source record or deleting its competing assertion. The query is an illustrative, unexecuted SPARQL design; check syntax and store support before relying on it.

`OPTIONAL` preserves a solution when the optional pattern has no match, leaving its variable unbound. If multiple ends or multiple matching assertions are present, the query can return multiple rows. That is useful evidence of multiplicity but can also duplicate apparent answers; impose appropriate constraints or aggregate deliberately, without suppressing distinct evidence. An omitted assertion fails the mandatory pattern and disappears from this particular result, so a separate validation or audit query is needed to find unsupported claims.

`FILTER NOT EXISTS` can ask whether a matching pattern is absent from the active dataset. It cannot prove that a source or relationship does not exist in the world. A query's dataset, default graph configuration, entailment settings, and access permissions all affect its results. State whether inferred triples are included and whether the selected release contains all relevant sources before treating absence as meaningful. SPARQL 1.1 defines graph patterns, optional matches, filters, datasets, and result forms; an implementation may add features whose behaviour should be documented separately.

9.3 Validate Graph Structure with Shapes Constraint Language (SHACL)

The Shapes Constraint Language (SHACL) expresses conditions over an RDF *data graph* in a separate *shapes graph*. A validator reports whether the chosen data conforms and identifies violations. For the local modelling rule that each applicability claim has one version, one project, and one start date, a compact shape is:

```
@prefix ex:  <https://example.org/northbridge/> .
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:ApplicabilityShape a sh:NodeShape ;
    sh:targetClass ex:ApplicabilityClaim ;
    sh:property [ sh:path ex:policyVersion ;
                  sh:minCount 1 ; sh:maxCount 1 ; sh:nodeKind sh:IRI ] ;
    sh:property [ sh:path ex:project ;
                  sh:minCount 1 ; sh:maxCount 1 ; sh:nodeKind sh:IRI ] ;
    sh:property [ sh:path ex:validFrom ;
                  sh:minCount 1 ; sh:maxCount 1 ; sh:datatype xsd:dateTime ] ;
    sh:property [ sh:path ex:validTo ;
                  sh:maxCount 1 ; sh:datatype xsd:dateTime ] ;
    sh:property [ sh:path ex:endKind ;
            sh:minCount 1 ; sh:maxCount 1 ; sh:in ( ex:BoundedEnd ex:UnboundedEnd ex:UnknownEnd ) ] ;
    sh:property [ sh:path ex:polarity ;
                  sh:minCount 1 ; sh:maxCount 1 ; sh:datatype xsd:boolean ] ;
    sh:property [ sh:path [ sh:inversePath ex:asserts ] ;
                  sh:minCount 1 ; sh:class ex:Assertion ] .

ex:AssertionShape a sh:NodeShape ;
    sh:targetClass ex:Assertion ;
    sh:property [ sh:path ex:sourceRecord ; sh:minCount 1 ; sh:nodeKind sh:IRI ] ;
    sh:property [ sh:path ex:locator ; sh:minCount 1 ] ;
    sh:property [ sh:path ex:supportedThrough ;
                  sh:maxCount 1 ; sh:datatype xsd:dateTime ] .
```

The inverse path finds assertions pointing to the claim; the second shape checks the assertion's source, locator and optional support horizon. These are deliberately local requirements, not universal rules for all RDF claims. The shape also needs conditional constraints: `BoundedEnd` requires exactly one `validTo` and no `supportedThrough`; `UnboundedEnd` requires neither; and `UnknownEnd` requires no `validTo` and exactly one assertion horizon not before `validFrom`. Express those alternatives with `sh:xone`, or with a validation query joining each assertion to its claim. Check the end/start ordering and horizon/start ordering in that same validation layer. Targeting matters: if a malformed claim lacks the type that makes it a target, this shape alone may never inspect it. The data graph supplied to validation must also include the relevant assertions; validating an isolated claim graph could report missing evidence that exists elsewhere. This shape is illus-

trative and unexecuted.

Additional checks should test that an end follows the start under the chosen interval convention, source identifiers resolve to retained records, and claims expected to be exclusive do not overlap for the same scope. The last check depends on domain semantics and may require SPARQL-based constraints or a separate validation query. Shape conformance tells us only that selected conditions passed for the selected graph. A fabricated register row with a well-formed date can conform, and a correct but incomplete extraction can fail. Preserve the validation report and the shapes version with the release, and investigate failures rather than silently dropping records. See the SHACL Recommendation for targets, constraint components, and validation reports.

9.4 Describe Production History with the PROV Ontology (PROV-O)

Provenance records how a representation was produced and by whom. The PROV Ontology (PROV-O) describes entities, activities, and agents, with relations between them. In this model, a retained register version is a `prov:Entity`, extraction is a `prov:Activity`, and the software or responsible organisation may be a `prov:Agent`. The resulting assertion is another entity. A minimal extension to the earlier Turtle is:

```
ex:r1 a prov:Entity .
ex:extract-r1 a prov:Activity ;
    prov:used ex:r1 ;
    prov:wasAssociatedWith ex:extractor .
ex:extractor a prov:SoftwareAgent .
ex:assertion-r1-p1 prov:wasGeneratedBy ex:extract-r1 ;
    prov:wasDerivedFrom ex:r1 .
```

This describes a possible extraction history; the example does not pretend that the activity has actually run. In a real release, identify the source version, software and configuration, run or review time, and responsible agent where known. If a person reviewed a candidate later, represent the review as a separate activity with its own inputs and outputs. The register's publisher, the person whose statement appears in minutes, the extractor, and the reviewer may all be different agents with different roles. PROV-O's qualified relations can attach roles or additional detail when a simple association is insufficient.

Provenance does not supply the authority rule. `prov:wasDerivedFrom` does not mean the register is correct, and `prov:wasAssociatedWith` does not identify an agent as the author of the original policy. Any authority rule must be recorded

separately and scoped to the relevant question; the executable uses abstention on conflicting evidence. Nor does extraction time replace valid time: an assertion generated on 20 March can describe applicability on 1 March. If a register correction is ingested later, keep the earlier source and release identifiable so that both a current answer and a historical answer can be reconstructed.

9.5 Property Graphs and Cypher

A property graph presents nodes connected by typed, directed relationships. Nodes may have labels and properties; relationships may also have properties. In Neo4j, a relationship has one type and a direction, and Cypher can match it in a directed or undirected pattern. Direction is part of the model even when a particular query ignores it. The Cypher manual describes matching patterns and the results produced by Neo4j.

An initial projection could represent the register claim as `(v1:PolicyVersion)-[:APPLIES_TO {validFrom: date('2024-01-01'), validTo: date('2024-04-01'), endKind: 'bounded', polarity: true}]->(atlas:Project)`. This keeps the claim qualifiers near the relationship. For an `unknown` end, retain `endKind: 'unknown'` and put `supportedThrough` on the source-specific assertion; for an `unbounded` end, retain `endKind: 'unbounded'` and no horizon. It can work for a view whose source and release are known and whose edge identity and evidence are maintained. However, if each source has a separate edge, the graph must distinguish two sources making the same qualified claim from two different claims. If the relationship stores only one source field, multiple locators or extraction histories may be awkward to represent without duplication or loss. If another entity needs to refer to the assertion, a claim/assertion node design is often clearer.

For a closer match to the RDF example, use `(:ApplicabilityClaim)` nodes connected by `:POLICY_VERSION` and `:PROJECT`, with dates on the claim; `(:Assertion)-[:ASSERTS]->(:ApplicabilityClaim)` and `(:Assertion)-[:FROM_SOURCE]->(:Source)` preserve the separate assertion. The following illustrative query retrieves the same register-backed result. It assumes native date properties, a single source node with `id: 'R1'`, and a locator on each assertion:

```
MATCH (claim:ApplicabilityClaim)-[:POLICY_VERSION]->(version:PolicyVersion),
      (claim)-[:PROJECT]->(project:Project {id: 'atlas'}),
      (assertion:Assertion)-[:ASSERTS]->(claim),
      (assertion)-[:FROM_SOURCE]->(source:Source {id: 'R1'})
```

```
WITH claim, version, assertion, source, project, date('2024-03-01') AS day
WITH claim, version, assertion, source, project, day,
     CASE
       WHEN day < claim.validFrom THEN 'outside'
       WHEN claim.endKind = 'bounded' AND day < claim.validTo THEN 'applicable'
       WHEN claim.endKind = 'bounded' THEN 'outside'
       WHEN claim.endKind = 'unbounded' THEN 'applicable'
     WHEN claim.endKind = 'unknown' AND day <= assertion.supportedThrough THEN 'applicable'
       WHEN claim.endKind = 'unknown' THEN 'indeterminate'
       ELSE 'invalid'
     END AS temporalStatus
RETURN claim.id AS claimId, version.id AS versionId,
       assertion.id AS assertionId, source.id AS sourceId,
       assertion.locator AS locator, claim.validFrom AS start,
       claim.validTo AS end, claim.endKind AS endKind,
       assertion.supportedThrough AS supportedThrough,
       claim.polarity AS polarity, temporalStatus
ORDER BY versionId, assertionId
```

This query filters on the designated register and returns `indeterminate` rather than treating a missing end as forever. Missing properties evaluate as null in relevant Cypher expressions; absence must not be mapped to `unbounded`. To show disagreement, run a separate query for the minutes and return its claim, temporal status and locator alongside the register's. To answer a two-hop path question, classify both premises and return the claim and assertion identifiers for *each* hop; the endpoints alone are insufficient evidence. Parameterise application inputs rather than constructing query strings from user-provided values. This Cypher is illustrative and unexecuted.

Cypher and SPARQL are query languages for different graph representations; Neo4j is a particular graph database product implementing Cypher. A property graph is not inherently Neo4j, and RDF is not inherently a particular vendor's triple store. Product versions and implementations differ in syntax, constraints, transaction guarantees, import options, security, and query planning. Select a documented version for any reproducible example. Graph-shaped syntax makes traversals legible, but it does not guarantee that a query is fast or that a path is semantically meaningful.

9.6 What Is Lost When a Graph Is Simplified?

The following comparison assumes that the starting records contain the identified claims, assertions, and source information described above. It compares mappings, not the inherent expressive power of RDF, property graphs, or relational tables. Each richer representation can preserve the relevant information if the mapping is designed and tested to do so.

Mapping or projection	Retained	Lost or made ambiguous without extra fields
Qualified claim plus separate assertions	Version, project, interval, end kind, polarity, assertion IDs, source, locator and support horizon; disagreement can remain visible.	Nothing among these listed fields if all are mapped faithfully; extraction history, validity convention and release identity still need explicit representation.
One direct version-to-project edge with dates	Endpoints and stored interval, if the edge also carries end kind and polarity.	Source, locator, independent assertions, support horizon and indeterminate status unless separately attached; an unknown end may be mistaken for an unbounded one.
One unqualified version-to-project edge	Endpoints and relation type.	Interval, conditions, assertion identity, provenance, polarity, and whether the relation is merely reported or accepted.
Set of unique subject–predicate–object triples	Distinct triple values.	Repeated identical occurrences cannot encode the number or independence of sources; dates and evidence disappear unless modelled in additional triples.

Mapping or projection	Retained	Lost or made ambiguous without extra fields
Simple analytical graph collapsing edges by endpoints	Connectivity for the selected projection.	Parallel claims and their predicates, periods, polarity, sources, and sometimes direction, depending on the projection.
Name-only person node for the minutes' Alex	The surface name.	The difference between a mention and an identified person; merging with either directory person would invent an identity decision.

Loss is relative to the questions. A connectivity-only graph may be suitable for a labelled exploratory figure, provided the release states its selection rules and links back to the detailed records. It is unsuitable as the only evidence for who asserted a dated relationship. A round trip should compare stable identifiers, typed values, intervals, end kinds, support horizons, assertion multiplicity, source locators, polarity, unresolved mentions, and release information, not merely node and edge counts. If a target cannot represent a field directly, retain a documented sidecar or declare the export lossy; never silently reconstruct an unknown end as unbounded or otherwise missing information as fact.

9.7 Storage and Workload Determine the Boundary

A graph model describes data and query meaning; a storage engine chooses physical structures to retrieve it. A table can store nodes and edges as rows with identifiers and use joins to follow paths. An RDF store may index permutations of subject, predicate, object and graph to find triples for different binding patterns. A property-graph engine may use indexes to find starting nodes and adjacency-oriented structures to expand relationships. These are common design approaches, not universal guarantees: engines vary, and a query planner may combine indexes, scans, joins, caching and traversal operators. Neo4j

documents multiple store formats, so do not infer current physical layout or performance from a generic description of an older release.

Starting-point selectivity matters. Finding Atlas by a stable indexed ID and expanding a small, bounded neighbourhood is a different workload from scanning all claims for a date range or aggregating across every policy. A high-degree node can make even a short traversal expensive; an unbounded path can produce a very large number of possible paths or encounter cycles. A relational query can be efficient for repeated fixed-depth joins, and a graph database can also scan and aggregate. Measure actual plans, cardinalities, cache conditions, ingestion costs, and result size rather than assuming that one physical model wins every query.

DuckDB's role in the Northbridge construction route is to hold and query the tabular records produced by Python: it supports joins, filtering and aggregation, and its SQL also supports recursive common table expressions for iterative queries. It is not necessary to add a server merely because the data represents a graph. A separate RDF serving layer becomes attractive when shared RDF identifiers, a SPARQL interface, RDF interchange, or SHACL validation are central requirements. A property-graph service becomes worth testing when interactive neighbourhood and path queries, graph-oriented application development, or a particular operational requirement justify it. Neo4j is one concrete choice within the latter category, not a synonym for a graph database.

Compare candidates using the *same* versioned data and questions. Include a register-scoped date query, a two-hop path returning evidence for every edge, a minutes disagreement query, and an unresolved-mention query. Add representative larger synthetic distributions only if they preserve useful properties such as degree skew and assertion multiplicity. Measure build and update time, storage, warm and cold query latency, throughput under expected concurrency, and the cost of exporting evidence. Fix hardware, versions, indexes, dataset, and result semantics; inspect plans and report failures as well as successful timings. A tiny Northbridge example tests correctness and integration, not production scalability. Include backup, access control, deployment, and skills in the decision, because a modest query-time improvement can be outweighed by the cost of maintaining another service.

9.8 A Bounded Future Neo4j Smoke Test

A later integration can test the mapping without making Neo4j a prerequisite for the canonical offline route. Use an isolated, single-node Neo4j Community container with an explicitly pinned image version or digest and a disposable database volume. Bind the service only as needed for the local test, use test-only credentials supplied through the test environment, and remove the test container and its disposable data after the run. Record the image, driver and Cypher versions and exact command in the repository's container workflow. This is a proposed test design, not an executed test or a production configuration.

Load the same frozen fixture through a documented mapping using stable application IDs for versions, projects, claims, assertions and sources. Preserve the original source locators, date values, polarity, and unresolved mentions; do not use internal database IDs as external identity. Create a uniqueness constraint on the selected stable IDs where supported by the pinned edition, and explicitly test required fields in the loader or validation query. A uniqueness constraint alone does not require every node to carry the property. Make loading idempotent under the chosen keys, and verify that a second load creates no extra nodes, relationships, or assertions. Consult the pinned version's constraint documentation for exact syntax and availability.

The smoke test has bounded acceptance cases:

1. Compare the loaded claim and assertion IDs, qualifiers, polarity, source IDs and locators with the frozen canonical export; fail on missing or extra records rather than merely comparing totals.
2. Query a dated applicability claim and a fixed two-hop path, returning the evidence for each hop. Compare ordered results with the canonical implementation under the *same* selection and conflict rules.
3. Return the minutes' competing assertion and its locator without deleting or silently resolving it; retain the fixture's unresolved Alex mention.
4. Reject or report an unknown endpoint, malformed interval, missing evidence, and duplicate stable ID. Test an end boundary on 1 April explicitly.
5. Apply a small, explicitly defined correction as a new source or assertion version, verify the changed current answer, and verify that the earlier release remains reconstructible. If release history is not yet implemented, mark this case blocked rather than claiming success.

Exercise only the fixed query patterns needed for these cases, using parameterised inputs and a timeout. Capture exit status and assertion failures, and clean up the test's own resources even on failure. This verifies mapping and basic operation on the fixture; it does not establish availability, security, disaster recovery, production throughput, or a performance advantage. Neither this design nor the illustrative Cypher above is evidence that Neo4j has been run.

10 Analyse a Projection in R

The companion `examples/r/knowledge_graph.R` hard-codes the fixture's eight entity identifiers and five positive-projection relations. It uses `igraph` to create a directed graph, and `tidygraph` to calculate node summaries. It does not read the JSON or query DuckDB, so changes to the fixture require the companion data to be kept in sync. This projection intentionally drops end kind, support horizon, polarity, provenance and claim identity; it is therefore undated and cannot implement the indeterminate answer policy. The igraph R documentation describes the graph operations.

The script asserts a directed shortest path from version 1 through Atlas to Northbridge, with total cost 2 under its unit weights. It also asserts three weakly connected components: the main connected group and the two isolated Alex entities. Weak connectivity ignores edge direction; the shortest path uses outgoing edges. Neither calculation establishes the authority, temporal validity or truth of the path. Weights here are assigned unit costs, not confidence scores.

The projection collapses the positive `c4` and `c6` assertions into one version-2 edge, discards `c5` and all dates, source IDs and locators, and retains `c1` even outside its validity interval. The R graph contains no claim IDs on edges, so a path cannot itself trace each step back to its evidence; return to the claims table or build an evidence-preserving mapping. A dated or source-selected analysis requires a different, explicitly defined projection.

11 Validate, Evaluate, and Diagnose

This section separates three questions that are often conflated: whether the records satisfy declared constraints, whether rules derive a conclusion, and

whether a source supports an assertion. It then turns those distinctions into evaluation cases and release decisions, so a passing database check cannot be mistaken for evidence that the graph is true or complete.

11.1 Validation, Inference and Verification

These operations answer different questions. **Validation** asks whether records satisfy declared constraints. The executable rejects duplicate IDs, unknown source and entity references, invalid date and end-kind combinations, invalid polarity or empty predicates, and a claim mention that fails the source-and-participant check. It also checks that the `relations` entries are unique, have positive weights and equal the set of positive claim triples. Its negative tests mutate copies of the fixture to exercise unknown entities, malformed temporal fields, a mismatched mention, a duplicate ID and a selected candidate on an abstention. The construction slice separately checks exact source spans, accepted/rejected/abstained identity decisions and a refused unsupported date phrase. Passing those checks says that these particular rules held for this input; it does not establish that the source text actually supports any claim.

The projection check compares positive triples, not dates, provenance, credibility or the numerical meaning of weights. As the R analysis illustrates, passing it does not make the projection a source-resolved, dated view.

Inference derives a conclusion from premises under an explicit rule. For example, a system could define a rule that turns an `applies_to`/`belongs_to` path into a carefully named organisational association. The current Python example only returns the path; it does not implement that rule or materialise a new claim. Before adding it, specify its meaning, acceptable predicates, scope, source restrictions and temporal overlap. If two half-open input intervals overlap, the derived relationship can hold at most during their intersection; if they do not overlap, the path is not evidence of a simultaneous relationship. Record both premise IDs, the rule version and the derivation run, so a corrected premise or rule can invalidate the result. A derived association is not the same as an explicit statement in the register.

Verification checks whether an assertion is supported by the appropriate evidence and is adequate for the question. A reviewer could locate the retained register version, inspect its Atlas entry, confirm the version and interval, and record the review and its limitations. Checking that `c1.source` refers to `s-`

`register` is validation; comparing `c1` with the actual register is verification. This synthetic fixture supplies metadata and claims rather than the complete underlying source documents, so it cannot demonstrate that external verification step. It can demonstrate the structure of a review workflow and the need to abstain when the available evidence conflicts.

For RDF data, SHACL specifies validation of a data graph against shapes and produces a validation report. It does not make a conforming graph factually correct. Likewise, the DuckDB schema above and the Python checks are not a SHACL implementation. If a system validates inferred data, record whether validation ran on asserted data only or on an expanded graph, and keep the underlying asserted data distinguishable from derived results.

11.2 Gold Sets and Useful Measures

Build an evaluation set from questions and independently adjudicated evidence, not merely from rows copied from the graph being tested. A useful gold item contains the question, entity and predicate scope, requested valid date, source boundary, expected answer or abstention, supporting and opposing evidence identifiers, and an explanation of the adjudication. Record who reviewed it, the evidence version and the policy version. The fixture's three date cases are good regression examples, but their expected outputs were chosen from the same synthetic input and policy; passing them does not estimate accuracy on unfamiliar documents.

For the conservative target policy, a small regression table is:

Question for version 1 and Atlas	Expected selected evidence	Expected policy output
31 December 2023	none	`abstain: no evidence`
15 March 2024	`c1` positive, `c5` negative; both applicable	`abstain: conflicting evidence`
16 March 2024	`c1` applicable, `c5` indeterminate	`abstain: indeterminate temporal scope`
1 April 2024	`c1` outside, `c5` indeterminate	`abstain: indeterminate temporal scope`
15 April 2024	`c1` outside, `c5` indeterminate	`abstain: indeterminate temporal scope`

Add a boundary case for 1 April: `c1` is excluded at its bounded end, while the unknown-ended `c5` is indeterminate after its support horizon. A dated two-hop case should classify both edge intervals; an outside premise makes the path outside, while an indeterminate premise makes the path indeterminate. A provenance case should check source, locator and mention, and that a missing mention such as `c7` does not eliminate the claim. The fixture records `m-alex` as unresolved and `r2` as abstaining between two candidates; the validator and assertions check this representation, but no entity-resolution algorithm is executed.

Separate stages in the scorecard. For extraction, compare proposed source assertions with independently reviewed assertions and count correct, missing and spurious records. For entity resolution, evaluate links and non-links, especially same-name people and changing names. For retrieval, assess whether all required supporting and opposing passages were returned. For final answers, measure correctness with respect to the specified evidence and policy, as well as whether the cited evidence actually supports each answer. A correct label with an unrelated citation is an evidence failure. Report results by slice, including ambiguous identities, negative claims, conflicts and temporal boundaries; aggregate accuracy can conceal the cases that motivate a knowledge graph.

Precision is correct returned items divided by returned items; recall is correct returned items divided by all gold items that should have been returned. Define the matching unit before calculating either: a claim ID, a source-qualified assertion, a date-qualified answer or an evidence span can lead to different counts. A system can improve apparent precision by answering less often, so report answer coverage and the correctness of answered cases alongside the number and reasons for abstentions. If the denominator is zero, report the count and mark the ratio undefined rather than inventing a value. For a small gold set, publish counts rather than implying that a percentage gives a precise population estimate.

Construct a held-out set from source records or document families not used to tune extraction and matching rules. Splitting individual rows from the same document across development and test can leak the same wording, entities and corrections into both. Review difficult cases independently, resolve disagreements using written criteria, and retain unresolved or disputed labels rather than forcing a false consensus. Include negative examples and missing-evidence cases, but label absence as negative only when the gold-set scope justifies that conclu-

sion. Re-run the same gold set after changing the schema, extraction pipeline, linking rules, answer policy or sources; investigate regressions before replacing expected answers.

12 Release, Update, and Govern

A release is a reproducible combination of input versions, transformations, schema and answer policy, not merely a database file. For the fixture, the inputs include `primer_knowledge_graph/examples/fixture.json`, `primer_knowledge_graph/examples/python/knowledge_graph.py`, the manifest and the primer-specific runtime definition. Record their revisions or digests, the runtime and dependency versions, validation results, evaluation results and the exact build command. The current example uses a fresh temporary DuckDB file and tests that it can be reopened; it does not itself create a release manifest, preserve input snapshots or publish a database. Those are production extensions, not features to infer from the example.

For an operational graph, ingest a new source version into a staging area; validate identifiers, references and intervals; compare the candidate with the previous release; run the evaluation set; and review material changes before publication. The diff should show added, changed and withdrawn assertions, affected entities and derived paths, changes in source coverage and the effect on representative questions. Build into a new versioned location rather than overwriting the only copy. Publish the version and its provenance together, and retain a tested route back to the prior release subject to retention and access rules. A reproducible build requires access to permitted input versions as well as code; a digest can identify missing material but cannot reconstruct it.

A correction, an end of validity and a retraction are different events. Ending `c1` at 1 April describes the asserted policy interval; it does not say that the register was wrong when it made the assertion. A later correction might revise that interval and require a new source version. A retraction says that an assertion or source should no longer be relied upon under a stated policy. Preserve a record of the change, its effective and recorded times, reason, actor and replacement when applicable. Whether the old content itself can be retained depends on legal, privacy and retention obligations; a tombstone or restricted audit record may be needed where the original must be removed. Do not silently delete an

assertion and leave derived results or cached answers looking unchanged.

Trace dependencies from a changed assertion to derived claims, projections, indexes and published answers. On retraction, recompute or invalidate those dependants and reassess answers that relied on them. A derived result with two independent supporting paths may survive one withdrawn premise, but only if the remaining path still satisfies the rule and evidence policy. This dependency tracking is a proposed production feature; the Python fixture does not maintain it. For mutable sources, keep the old source version and locator where permitted so that a reviewer can explain what a previous release knew, even if the preferred answer has since changed.

Assign named responsibility for source admission, identity resolution, schema and predicate definitions, conflict policy, release approval and incident response. A data steward need not perform every task, but decisions require an accountable owner and a documented escalation route. Restrict access to sensitive sources and evidence spans, avoid publishing private attributes merely because they can be joined, and check whether downstream users can infer sensitive information from paths. Govern identifiers, corrections and deletion across replicas and caches, not only in the primary database. Review retention, permissions and intended use with the relevant owners before sharing a release.

Finally, communicate the graph's boundary with each release: which sources and dates it covers, which claims are asserted or derived, how conflicting evidence is presented, which validation and evaluation checks passed, and where a reader can report a suspected error. A reliable graph is one whose answers can be challenged and repaired; the presence of a graph database or a successful schema check is only one part of that process.

For this milestone, the omitted pieces are intentional: Neo4j execution remains a future pinned-container smoke test; the current Python example does not implement a full source extractor or update/retraction engine; the R companion remains a manually synchronised positive projection; and the synthetic fixture cannot support a research-grade coverage or accuracy study. No figures are required to make the model executable, so the accessible construction diagram and comparison tables carry that explanatory burden. These are scope dispositions, not claims that the omitted capabilities are complete.

13 Further Reading

- RDF 1.1 Concepts and Abstract Syntax defines RDF graphs, triples, identifiers, literals, and the distinction between syntax and model.
- Defining N-ary Relations on the Semantic Web explains patterns for relations that need more than two participants or qualifiers.
- PROV-O provides vocabulary for describing provenance across entities, activities, and agents.
- SPARQL 1.1 Query Language and SHACL specify RDF graph querying and shape-based validation, respectively.
- Estimating the False Negatives Due to Blocking in Record Linkage examines candidate-selection error, a distinct source of identity-resolution failure.
- DuckDB documentation and the Neo4j Cypher manual provide implementation details that must be checked against the versions actually pinned for examples.